Agent 개념과 멀티턴 대화 (실습)

5-1. 챗봇과 Agent의 차이
5-2. 이번 과정의 최종 그림
5-3. Message History로 멀티턴 대화 (실습)

5-1. 챗봇과 Agent의 차이

> 모델이 질문에 따라 도구를 스스로 고르는 것을 직접 확인해봅니다.

지금까지 다루었던 실습은 전부 단방향으로, 입력에서 출력으로 한 번 흘러가고 끝납니다.

Agent는 다릅니다. Agent의 핵심은 tool calling 루프입니다.

- **생각(Thought)**: 무엇을 해야 할지 판단합니다.
- **행동(Action)**: 필요한 도구를 호출합니다.
- **관찰(Observation)**: 도구가 돌려준 결과를 확인합니다.

이 사이클을 목표를 달성할 때까지 반복합니다. 이 패턴을 정립한 것이 ReAct(Reasoning + Acting)입니다.

오늘은 이 루프의 첫 절반, 즉 모델이 도구 호출을 결정하는 것까지만 봅니다.
결정을 받아 실제로 함수를 실행하고 결과를 모델에 돌려주는 루프는 3일차 LangGraph ReAct Agent에서 완성합니다.

따라하기

1. `@tool` 데코레이터(Java 어노테이션에 대응하는 개념)로 함수를 도구로 등록하고 모델이 질문에 따라 도구를 스스로 고르는 것을 확인합니다. 아래 코드를 `day1_practice/tool_decision.py` 로 저장하고 실행합니다.

```python
# tool_decision.py - 모델의 도구 호출 "결정" 확인
from dotenv import load_dotenv
from langchain_core.tools import tool
from langchain_aws import ChatBedrockConverse

load_dotenv()

@tool
def multiply(a: int, b: int) -> int:
    """두 정수 a와 b를 입력받아 곱을 반환한다. 정확한 산술 계산이 필요할 때 사용."""
    return a * b

@tool
def search_employee(name: str) -> str:
    """삼성SDS 임직원 디렉터리에서 이름으로 직원 정보를 조회한다."""
    # 교육용 더미 데이터입니다
    fake_db = {
        "김하늘": "김하늘 / 클라우드운영팀 / haneul.kim@samsungsds.example.com",
        "박도윤": "박도윤 / 물류플랫폼팀 / doyun.park@samsungsds.example.com",
    }
    return fake_db.get(name, f"{name}을(를) 찾을 수 없습니다.")

# bind_tools: 모델에게 "이런 도구들이 있다"고 알려줍니다
llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
)
llm_with_tools = llm.bind_tools([multiply, search_employee])

queries = [
    "1234567 곱하기 8901234 계산해줘.",              # 단일 도구
    "김하늘 찾아서 이메일 알려주고 15 곱하기 25도.",  # 복합 (도구 2개)
    "그냥 안녕이라고만 인사해줘.",                    # 도구 불필요
]

for q in queries:
    response = llm_with_tools.invoke(q)
    print(f"[질문] {q}")
    if response.tool_calls:  # 모델이 호출하기로 "결정"한 도구 목록 (아직 실행 전)
        for call in response.tool_calls:
            print(f"  -> 호출 결정: {call['name']}({call['args']})")
    else:
        print(f"  -> 직접 응답: {response.content}")
    print()
```

관찰 포인트는 세 가지입니다.
모델이 질문을 분석해 도구를 스스로 고릅니다.

하나의 질문에서 두 도구를 동시에 호출하기로 결정할 수도 있습니다. 도구가 필요 없으면 호출하지 않고 직접 답합니다.

이때 docstring(함수 설명 문자열)이 곧 모델에게 전달되는 도구 설명이므로 언제 이 도구를 쓰는지까지 담아 자세히 쓰는 것이 원칙입니다.

막힐 때

- `tool_calls`가 늘 비어 있습니다: docstring이 없거나 너무 부실하면 모델이 도구의 용도를 몰라 선택하지 못합니다.
- 도구를 호출 결정했는데 결과가 안 나옵니다: 정상입니다. 오늘 코드는 결정까지만 봅니다. 실행과 재판단 루프는 3일차에서 만듭니다.
- `@tool`을 붙였더니 일반 함수처럼 호출이 안 됩니다: 데코레이터가 함수를 Tool 객체로 바꿉니다. 직접 실행해 보려면 `multiply.invoke({"a": 3, "b": 5})`처럼 씁니다.

확인: 세 질문에서 단일 도구 호출, 도구 2개 호출, 직접 응답이 각각 나오면 성공입니다.

> 모델이 입력에 따라 도구를 스스로 고른다는 걸 확인했습니다.

5-2. 이번 과정의 최종 그림

> 7일차에 완성할 Multi-Agent 프로토타입의 전체 구조를 미리 이해해봅니다.

7일차에 완성할 최종 산출물은 삼성SDS 사내 어시스턴트 Multi-Agent 프로토타입입니다. 직원이 이렇게 묻는 상황을 상상합니다.

"신입 사원 온보딩 가이드 요약해 주고, 지난 분기 우리 팀 프로젝트 비용 알려줘. 그리고 예산 증액 절차는 어떻게 돼?"

이 한 질문을 처리하려면 문서 검색(RAG), DB 조회(MCP 도구), 위험 작업 승인(HITL), 여러 전문 Agent의 협업(Supervisor)이 전부 필요합니다. 매일 부품을 하나씩 더해 최종 그림을 완성합니다.

오늘 배운 것과의 연결이 핵심입니다. 오늘의 체인은 최종 시스템의 가장 안쪽 부품입니다. RAG도 결국 검색 결과를 프롬프트에 끼워 넣은 체인이고 Agent의 각 단계도 결국 모델 `invoke` 입니다.

> 오늘의 체인이 최종 Multi-Agent의 기본 부품이라는 전체 그림을 확인했습니다.

5-3. Message History로 멀티턴 대화 (실습)

> 대화 기억이 유지되는 체인을 만들어봅니다.

먼저 턴(turn)이 무엇인지부터 정리합니다. 사용자가 한 번 말하고 모델이 한 번 답하는 한 쌍이 한 턴입니다.

구분	설명	예
싱글턴	매 요청이 서로 독립이라 앞 대화를 몰라도 된다	번역, 분류, 요약, 공지 문구 생성
멀티턴	앞 대화를 알아야 답할 수 있다	상담, 되묻기, "그거 다시", 이전 답변 수정 요청

4번째 세션까지 만든 체인은 전부 싱글턴이었습니다. 공지 문구를 쓰든 문의를 분류하든, 직전에 무슨 요청이 있었는지 알 필요가 없었습니다. 하지만 헬프데스크 상담처럼 되묻고 답하는 서비스라면 앞 턴을 기억해야 합니다.

2번째 세션 2-4에서 확인했듯 LLM은 stateless입니다. 한 번 답하고 나면 방금 무슨 말을 했는지 서버 쪽에 남지 않습니다. 그래서 멀티턴은 모델이 기억해서 되는 게 아니라, **우리가 과거 대화를 매번 함께 보내서** 만드는 것입니다.
웹으로 치면 HTTP가 stateless라 세션과 쿠키로 로그인 상태를 유지하는 것과 같은 구조입니다.

멀티턴 대화의 원리는 단순합니다.

- **저장(Store)**: 대화를 어딘가에 보관합니다.
- **조회(Retrieve)**: 새 질문이 오면 과거 대화를 꺼냅니다.
- **결합(Combine)**: 과거와 현재를 함께 모델에 보냅니다.

따라하기

1. 먼저 리스트를 직접 관리하며 원리를 확인합니다. 아래 코드를 `day1_practice/multiturn_manual.py` 로 저장하고 실행합니다.

```
# multiturn_manual.py - chat_history 리스트 직접 관리로 원리 확인
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse
from langchain_core.messages import HumanMessage

load_dotenv()
llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
)

chat_history = []  # 대화 기록을 우리가 직접 들고 있습니다

for user_input in ["안녕, 내 이름은 김하늘이야.", "내 이름이 뭐였지?"]:
    chat_history.append(HumanMessage(content=user_input))
    response = llm.invoke(chat_history)   # 기록 전체를 매번 보냅니다
    chat_history.append(response)         # AI 응답도 기록에 추가합니다
    print(f"사용자: {user_input}")
    print(f"AI: {response.content}")
    print(f"(현재 기록 {len(chat_history)}개, 입력 토큰 {response.usage_metadata
['input_tokens']})\n")
```

2. LangChain의 권장 방식으로 자동화합니다. `session_id` 로 사용자별 대화 기록이 자동 분리됩니다. 아래 코드를 `day1_practice/multiturn_history.py` 로 저장하고 실행합니다.

```
# multiturn_history.py - 권장 방식: RunnableWithMessageHistory
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse
from langchain_community.chat_message_histories import ChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

load_dotenv()

# 세션별 대화 기록 저장소 (실제 서비스에서는 Redis나 DB로 교체합니다)
_store: dict[str, ChatMessageHistory] = {}

def get_session_history(session_id: str) -> ChatMessageHistory:
    """세션 ID별 기록 객체를 반환. 함수 시그니처만 유지하면 저장소는 무엇이든 됩니다."""
    if session_id not in _store:
        _store[session_id] = ChatMessageHistory()
    return _store[session_id]

model = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
)

# 모델에 메모리를 결합: 기록 저장과 결합이 자동으로 처리됩니다
with_memory = RunnableWithMessageHistory(model, get_session_history)

# session_id로 사용자를 구분합니다
config = {"configurable": {"session_id": "user_123"}}
print("[1차]", with_memory.invoke("안녕, 내 이름은 김하늘이야.", config=config).con
tent)
print("[2차]", with_memory.invoke("내 이름이 뭐였지?", config=config).content)

# 다른 세션은 격리됩니다: 같은 체인, 다른 기억
other = {"configurable": {"session_id": "user_456"}}
print("[다른 사용자]", with_memory.invoke("내 이름 알아?", config=other).content)
```

코드의 `_store: dict[str, ChatMessageHistory] = {}` 에서 앞의 밑줄은 "이 모듈 안에서만 쓰는 내부 변수"라는 파이썬의 관례 표시입니다.

Java의 `private` 처럼 강제하는 장치는 없고 약속입니다. 콜론 뒤는 타입 힌트로 문자열을 키로 하고 `ChatMessageHistory` 를 값으로 갖는 dict라는 뜻이며, 실행 자체에는 영향을 주지 않습니다.

막힐 때

- `config`를 빼먹으면 오류가 납니다: 어느 세션의 기록을 쓸지 알아야 하므로 `session_id`가 필수입니다. `config={"configurable": {"session_id": "..."}}` 형태를 그대로 익혀 둡니다.
- 프로그램을 다시 실행하면 기억이 사라집니다: `_store`는 메모리 dict라 프로세스가 죽으면 사라집니다. 운영에서는 `get_session_history` 안에서 Redis나 DB를 조회하도록 갈아 끼웁니다.
- `langchain_community` import 오류: `pip install langchain-community`로 설치합니다.

확인: user_123 세션은 이름을 기억하고 user_456 세션은 모르면 성공입니다.

> 기억은 모델이 아니라 애플리케이션이 들고 있다는 원리를 익혔습니다.
